

Data Structure & Algorithms

Program Efficiency & Complexity Analysis

Algorithm

- An algorithm is a procedure used for solving a problem or performing a computation.

A finite set of statements that guarantees an optimal solution in a finite interval of time.

Good Algorithm?

Run in Less Time

Consume Less Memory

Measuring Efficiency

- The efficiency of an algorithm is a measure of the amount of resources consumed in solving a problem of size n .
- The most obvious way to measure the efficiency of an algorithm is to run it and measure how much processor time is needed.

Running Time of an Algorithm

Depends upon

- Input size
- Nature of Input

The efficiency is measured with the help of asymptotic notations.

An algorithm may not have the same performance for different types of inputs. With the increase in the input size, the performance will change.

The study of change in performance of the algorithm with the change in the order of the input size is defined as asymptotic analysis.

Asymptotic Notations

- Asymptotic notations are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value.

There are mainly three asymptotic notations:

- Big-O notation
- Omega notation
- Theta notation

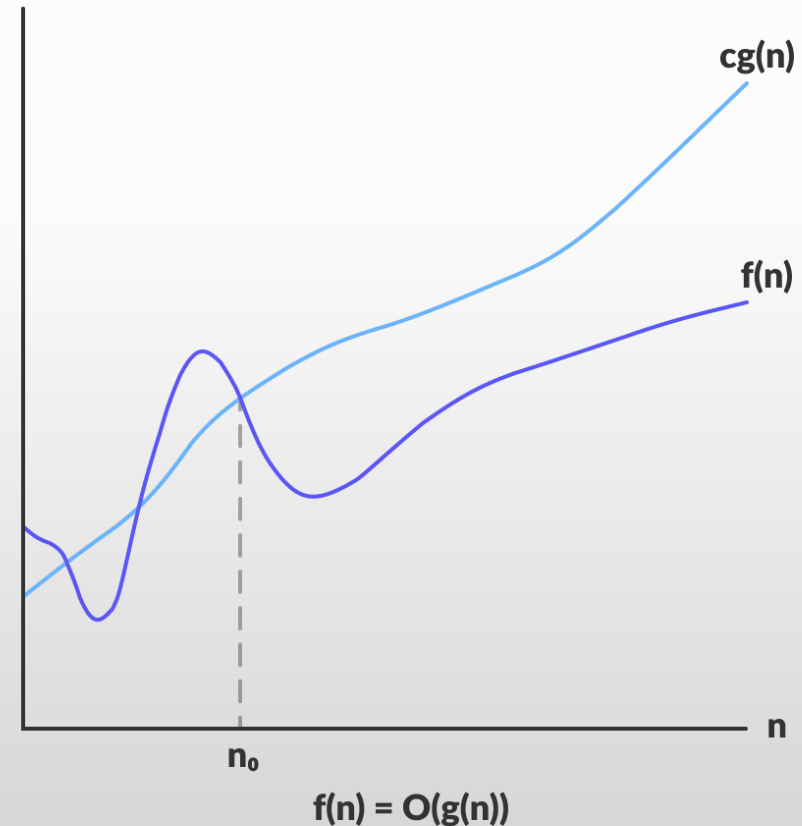
Big-O Notation (O-notation)

- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.

$O(g(n)) = \{ f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0 \}$

The above expression can be described as a function $f(n)$ belongs to the set $O(g(n))$ if there exists a positive constant c such that it lies between 0 and $cg(n)$, for sufficiently large n .

For any value of n , the running time of an algorithm does not cross the time provided by $O(g(n))$.



Big-O Notation (O-notation)

For example:

If $f(n)$ and $g(n)$ are the two functions defined for positive integers, then $f(n) = O(g(n))$ as $f(n)$ is **big oh of $g(n)$** or $f(n)$ is on the order of $g(n)$ if there exists constants c and n_0 such that:

$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

This implies that $f(n)$ does not grow faster than $g(n)$, or $g(n)$ is an upper bound on the function $f(n)$. In this case, we are calculating the growth rate of the function which eventually calculates the worst time complexity of a function, i.e., how worst an algorithm can perform.

The idea of using big o notation is to give an upper bound of a particular function, and eventually it leads to give a worst-time complexity. It provides an assurance that a particular function does not behave suddenly as a quadratic or a cubic fashion, it just behaves in a linear manner in a worst-case

Big-O Notation (O-notation)

Example 1: $f(n)=2n+3$, $g(n)=n$

Now, we have to find **Is $f(n)=O(g(n))$?**

To check $f(n)=O(g(n))$, it must satisfy the given condition:

$$f(n) \leq c \cdot g(n)$$

First, we will replace $f(n)$ by $2n+3$ and $g(n)$ by n .

$$2n+3 \leq c \cdot n$$

Let's assume $c=5$, $n=1$ then

$$2 \cdot 1 + 3 \leq 5 \cdot 1$$

$$5 \leq 5$$

For $n=1$, the above condition is true.

If $n=2$

$$2 \cdot 2 + 3 \leq 5 \cdot 2$$

$$7 \leq 10$$

For $n=2$, the above condition is true.

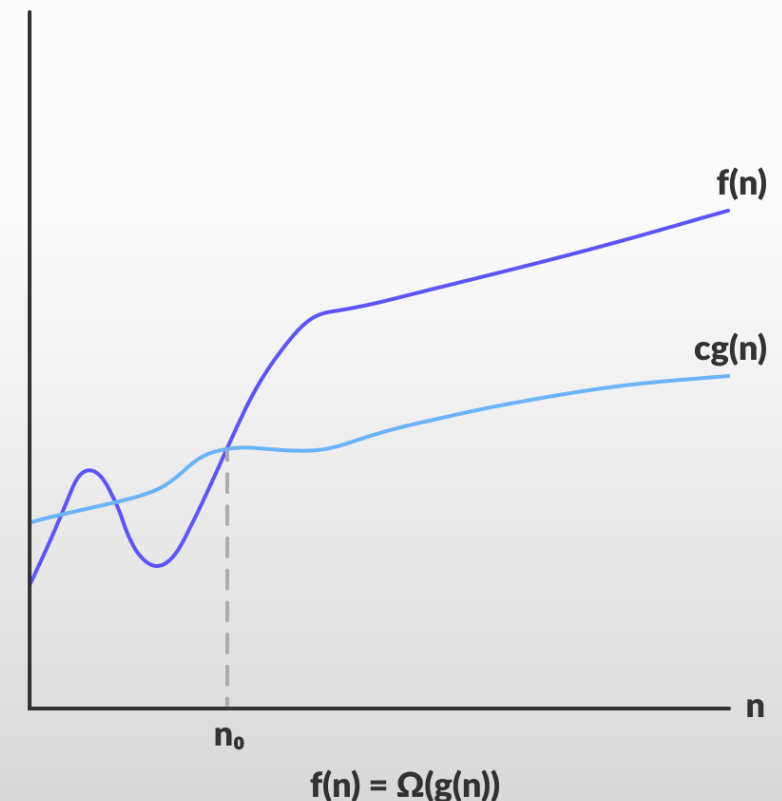
Omega Notation (Ω -notation)

- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best-case complexity of an algorithm.

$$\Omega(g(n)) = \{ f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \}$$

The above expression can be described as a function $f(n)$ belongs to the set $\Omega(g(n))$ if there exists a positive constant c such that it lies above $cg(n)$, for sufficiently large n .

For any value of n , the minimum time required by the algorithm is given by Omega $\Omega(g(n))$.



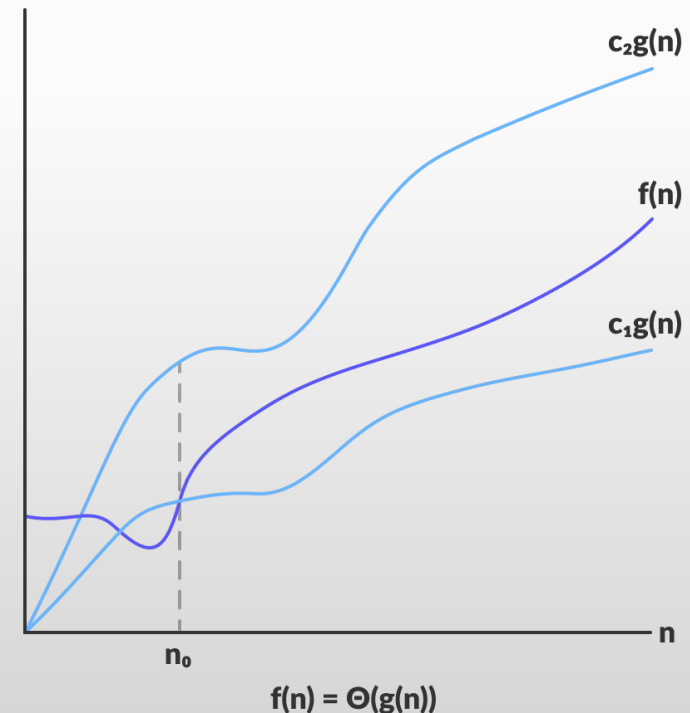
Theta Notation (Θ -notation)

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

$$\Theta(g(n)) = \{ f(n): \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0 \}$$

The above expression can be described as a function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1g(n)$ and $c_2g(n)$, for sufficiently large n .

If a function $f(n)$ lies anywhere in between $c_1g(n)$ and $c_2g(n)$ for all $n \geq n_0$, then $f(n)$ is said to be asymptotically tight bound.

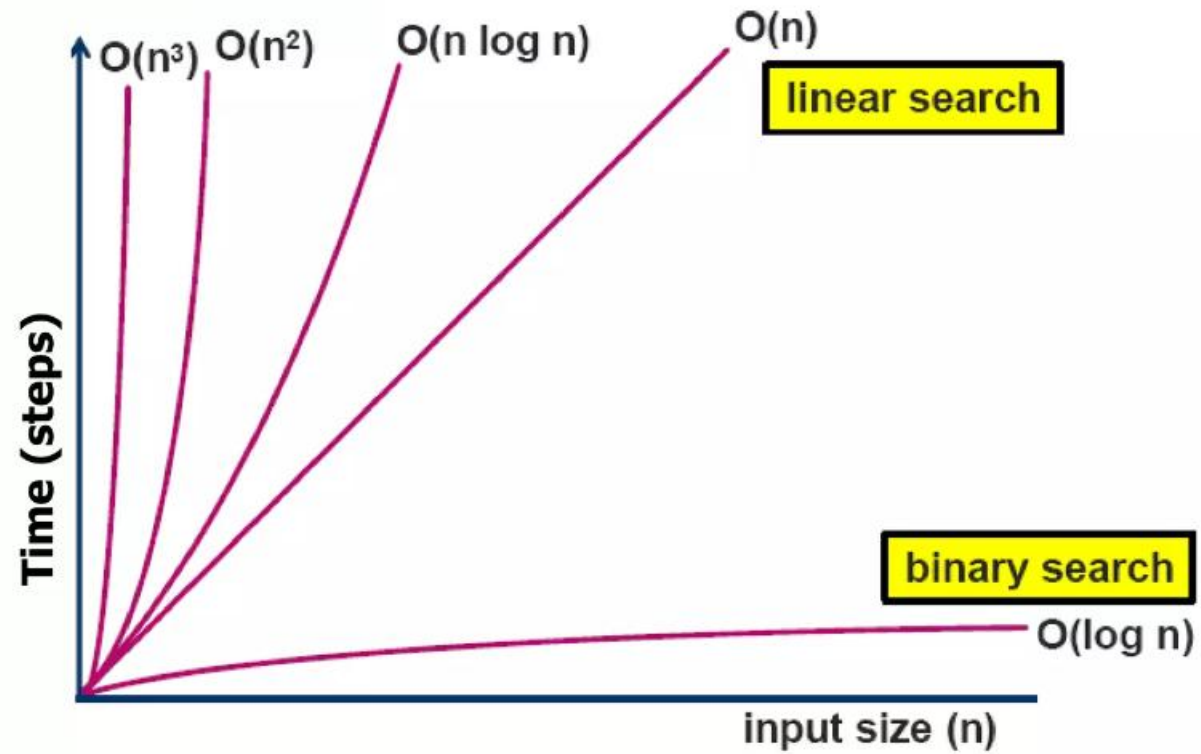


What does it mean?

- If $f(n) = \Theta(g(n))$ we say that $f(n)$ and $g(n)$ grow at the same rate , asymptotically
- If $f(n) = O(g(n))$ and $f(n) \neq \Omega(g(n))$, then we say that $f(n)$ is asymptotically slower growing than $g(n)$.
- If $f(n) = \Omega(g(n))$ and $f(n) \neq O(g(n))$, then we say that $f(n)$ is asymptotically faster growing than $g(n)$.

We generally like to express our algorithm as big O as we would like to know the upper bound of our algorithms.

Complexity Classes



Performance Classification

$f(n)$	Classification
1	Constant: run time is fixed, and does not depend upon n . Most instructions are executed once, or only a few times, regardless of the amount of information being processed
$\log n$	Logarithmic: when n increases, so does run time, but much slower. Common in programs which solve large problems by transforming them into smaller problems. Exp : binary Search
n	Linear: run time varies directly with n . Typically, a small amount of processing is done on each element. Exp: Linear Search
$n \log n$	When n doubles, run time slightly more than doubles. Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions. Exp: Merge
n^2	Quadratic: when n doubles, runtime increases fourfold. Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop). Exp: Insertion Search
n^3	Cubic: when n doubles, runtime increases eightfold. Exp: Matrix
2^n	Exponential: when n doubles, run time squares. This is often the result of a natural, “brute force” solution. Exp: Brute Force. Note: $\log n, n, n \log n, n^2 \gg$ less Input $\gg$ Polynomial $n^3, 2^n \gg$ high input $\gg$ non polynomial

Standard Analysis Techniques

- Constant Time Statements
- Analysing Loop
- Analysing Nested Loop
- Analysing Sequence of Statements
- Analysing Conditional Statements

Pointer

Pointer is used to points the address of the value stored anywhere in the computer memory. To obtain the value stored at the location is known as dereferencing the pointer. Pointer improves the performance for repetitive process such as

- Traversing String
- Lookup Tables
- Control Tables
- Tree Structures

Pointer Details

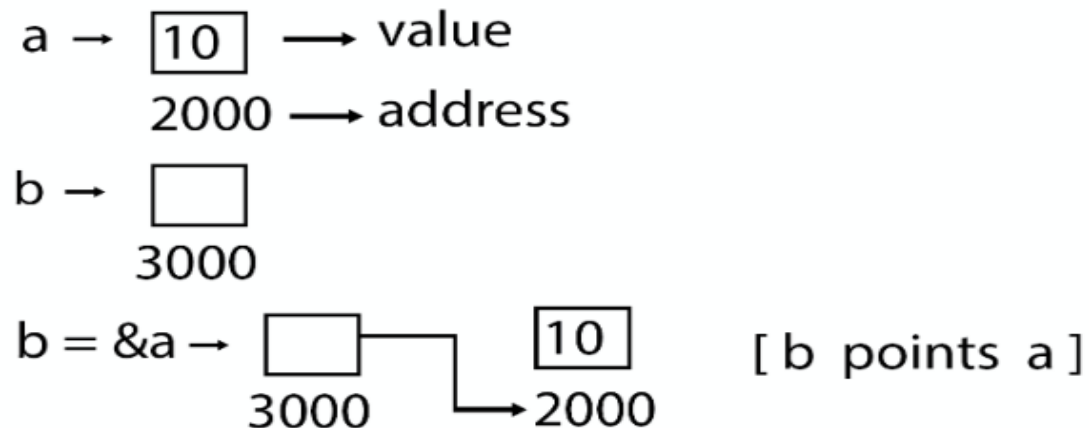
Pointer arithmetic: There are four arithmetic operators that can be used in pointers:
++, --, +, -

Array of pointers: You can define arrays to hold a number of pointers.

Pointer to pointer: C allows you to have pointer on a pointer and so on.

Passing pointers to functions in C: Passing an argument by reference or by address enable the passed argument to be changed in the calling function by the called function.

Return pointer from functions in C: C allows a function to return a pointer to the local variable, static variable and dynamically allocated memory as well.



```
#include <stdio.h>

int main( )
{
    int a = 5;
    int *b;
    b = &a;
    printf ("value of a = %d\n", a);
    printf ("value of a = %d\n", *(&a));
    printf ("value of a = %d\n", *b);
    printf ("address of a = %u\n", &a);
    printf ("address of a = %d\n", b);
    printf ("address of b = %u\n", &b);
    printf ("value of b = address of a = %u", b);
    return 0;
}
```

What does the following program print?

```
#include <stdio.h>

void f (int *p, int *q)
{
    p = q; *p = 2;
}

int i = 0, j = 1;

int main ( )
{
    f(&i, &j);
    printf ("%d %d \n", i, j);
    return 0;
}
```

